

Дизайн и анализ на алгоритми упражнение 8

Станислав Игов (Стъката)

2025/2026

Минимални покриващи дървета. Алгоритъм на Prim. Алгоритъм на Kruskal. Задачи.

Покриващо дърво (Spanning Tree)

Дефиниция

Нека $G = (V, E)$ е **свързан неориентиран граф**. **Покриващо дърво** (spanning tree) на G е подграф $T = (V, E_T)$, такъв че:

- T съдържа **всички** върхове на G .
- T е **дърво** - свързан и ацикличен граф.
- $E_T \subseteq E$ - ребрата на T са подмножество на ребрата на G .

Свойства на дърво с n върха:

- Има точно $n - 1$ ребра.
- Между всеки два върха има точно **един** път.
- Ако добавим ребро, се създава точно един цикъл.
- Ако премахнем ребро, дървото се разпада на две компоненти.

Пример: За граф с $V = \{1, 2, 3, 4\}$ и $E = \{\{1, 2\}, \{1, 3\}, \{2, 3\}, \{3, 4\}\}$, едно покриващо дърво е $E_T = \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$.

Минимално покриващо дърво (Minimum Spanning Tree)

Дефиниция

Нека $G = (V, E, w)$ е **свързан, претеглен, неориентиран граф**, където $w : E \rightarrow \mathbb{R}$ задава тегло на всяко ребро. **Минимално покриващо дърво** (MST) е покриващо дърво $T = (V, E_T)$, такова че общото тегло е минимално:

$$w(T) = \min \sum_{e \in E_T} w(e)$$

MST не е непременно единствено - може да има няколко покриващи дървета с еднакво минимално тегло.

Свойство на среза (Cut Property)

Основното свойство, на което се базират и двата алгоритъма:

Теорема (Cut Property): Нека $(S, V \setminus S)$ е произволен **срез** на графа (разделяне на върховете на две непразни множества). Ребро e , което пресича среза и има **минимално тегло** сред всички ребра пресичащи среза, принадлежи на някое MST.

Интуиция: Ако трябва да свържем двете страни на среза, най-изгодно е да използваме най-евтиното ребро.

Алгоритъм на Prim

[Линк към видео което обяснява как работи](#)

Идея

Алгоритъмът на Prim построява MST като **расте дърво** от един начален връх. На всяка стъпка добавяме **най-евтиното ребро**, което свързва връх от дървото с връх извън дървото.

Можем да мислим за алгоритъма така: поддържахме множество S от върхове, които вече са в дървото. На всяка стъпка разглеждаме среза $(S, V \setminus S)$ и добавяме ребро с минимално тегло, което пресича среза.

Псевдокод

```
Prim(G(V, E, w), start)
@1  MST_edges <- {}
@2  InTree[1..n] <- {false..false}
@3  Key[1..n] <- {INF..INF}           // минимална цена за достигане
@4  Parent[1..n] <- {NIL..NIL}
@5  Key[start] <- 0
@6  PQ <- MinPriorityQueue()          // приоритетна опашка
@7  PQ.push(start, 0)
@8
@9  while not PQ.empty do
@10     u <- PQ.pop()
@11     if InTree[u] then continue
@12     InTree[u] <- true
@13     if Parent[u] != NULL then
@14         MST_edges <- MST_edges + {(Parent[u], u)}
@15     end if
@16     forall (v, weight) in adj(u) do
@17         if not InTree[v] and weight < Key[v] then
@18             Key[v] <- weight
@19             Parent[v] <- u
@20             PQ.push(v, weight)
@21         end if
@22     end for
@23 end while
@24 return MST_edges
```

Коректност

На всяка стъпка алгоритъмът избира най-евтиното ребро, пресичащо среза $(S, V \setminus S)$, където S е множеството от вече добавените върхове. По свойството на среза (Cut Property), това ребро принадлежи на някое MST. С индукция по стъпките: след k стъпки избраните k ребра са подмножество на ребрата на някое MST. След $n - 1$ стъпки имаме пълно покриващо дърво, което е минимално.

Времева сложност

Всеки връх се извлича от приоритетната опашка най-много веднъж. Всяко ребро може да предизвика една операция `insert` в приоритетната опашка. С двоична пирамида (binary heap):

- `pop`: $O(\log |V|)$, извиква се $O(|V|)$ пъти.
- `push`: $O(\log |V|)$, извиква се $O(|E|)$ пъти.

$$Time(Prim) \in O((|V| + |E|) \cdot \log |V|)$$

За свързан граф $|E| \geq |V| - 1$, така че може да се запише и като $O(|E| \cdot \log |V|)$.

Алгоритъм на Kruskal

[Линк към видео което обяснява как работи](#)

Идея

Алгоритъмът на Kruskal построява MST като **сортира всички ребра** по тегло и ги добавя едно по едно, **пропускаяки тези, които създават цикъл**. За бързо проверяване дали ребро създава цикъл, използваме структурата **Union-Find** (Disjoint Set Union).

Union-Find (DSU)

Структура от данни, която поддържа разбиване на множество от елементи на непресичащи се подмножества. Поддържа две операции:

- **Find(x)** - връща представителя (корена) на множеството, съдържащо x .
- **Union(x, y)** - обединява множествата, съдържащи x и y .

С оптимизации **path compression** и **union by rank**, и двете операции работят в амортизирано $O(\alpha(n))$ време, където α е обратната функция на Акерман - на практика константа.

Псевдокод

```
Kruskal(G(V, E, w))
@1 MST_edges <- {}
@2 sort E by weight in ascending order
@3 DSU <- DisjointSetUnion(V)      // всеки връх е в свое множество
@4
@5 forall (u, v, weight) in E do    // в нарастващ ред по тегло
@6     if DSU.Find(u) != DSU.Find(v) then
@7         MST_edges <- MST_edges + {(u, v, weight)}
@8         DSU.Union(u, v)
@9         if |MST_edges| = |V| - 1 then
@10             break              // дървото е пълно
@11     end if
@12 end if
@13 end for
@14 return MST_edges
```

Коректност

Разглеждаме ребрата в нарастващ ред по тегло. Когато добавяме ребро (u, v) , то свързва два различни компонента. Нека S е компонентата на u . Тогава (u, v) е ребро, пресичащо среза $(S, V \setminus S)$, и тъй като разглеждаме ребрата в нарастващ ред, то е най-евтиното ребро пресичащо този срез. По Cut Property, то принадлежи на някое MST. Ребра, които свързват върхове от една и съща компонента, биха създали цикъл и се пропускат. След $n - 1$ добавени ребра имаме покриващо дърво, което е минимално.

Времева сложност

- Сортиране на ребрата: $O(|E| \cdot \log |E|) = O(|E| \cdot \log |V|)$, тъй като $|E| \leq |V|^2$.
- Обработка на ребрата с Union-Find: $O(|E| \cdot \alpha(|V|))$ - практически $O(|E|)$.

$$Time(Kruskal) \in O(|E| \cdot \log |V|)$$

Сравнение на Prim и Kruskal

	Prim	Kruskal
Подход	Расте дърво от връх	Сортира и добавя ребра
Структура от данни	Приоритетна опашка	Union-Find
Сложност	$O(E \cdot \log V)$	$O(E \cdot \log V)$
По-добър при гъсти графи	Да	Не
По-добър при разреждени графи	Не	Да

Задачи с MST

Задача 1: Минимална цена за свързване на точки

Условие: Дадени са n точки в равнината с координати (x_i, y_i) . Трябва да свържем всички точки с минимална обща цена, където цената на свързване на две точки i и j е **Манхатън разстоянието**: $|x_i - x_j| + |y_i - y_j|$. Всяка двойка точки може да бъде свързана.

Дадени са n точки в равнината с координати (x_i, y_i) . Трябва да свържем всички точки с минимална обща цена, където цената на свързване на две точки i и j е Манхатън разстоянието:

$$|x_i - x_j| + |y_i - y_j|.$$

Всяка двойка точки може да бъде свързана.

Вход: Масив от n точки $P[1..n]$, където $P[i] = (x_i, y_i)$.

Изход: Минималната обща цена за свързване на всички точки.

Идея

Всяка двойка точки може да бъде свързана, значи имаме пълен граф с n върха и $\binom{n}{2}$ ребра. Теглото на ребро (i, j) е Манхатън разстоянието. Търсим MST на този пълен граф.

Стъпки

1. Изчисляваме разстоянията между всички двойки точки - това са $\binom{n}{2} = \frac{n(n-1)}{2}$ ребра. Сложност: $O(n^2)$.
2. Прилагаме алгоритъм за MST (например Kruskal) върху получения пълен граф. Сложност на Kruskal: $O(|E| \cdot \log |E|) = O(n^2 \cdot \log n)$, тъй като $|E| = O(n^2)$.
3. Сумата от теглата на ребрата в полученото MST е отговорът.

Коректност

Задачата изисква да свържем всички n точки с минимална обща цена - това е точно дефиницията на MST. Построяваме пъления граф (защото всяка двойка точки може да бъде свързана) и намираме неговото MST.

Времева сложност

$$\text{Time} \in O(n^2 \cdot \log n)$$

Доминирана от сортирането на $O(n^2)$ ребра в Kruskal. Забележка: може да се постигне $O(n^2)$ с Prim без приоритетна опашка (с масив), тъй като при пълен граф линейното търсене на минимум в масив е по-ефективно от операции с heap.

Задача 2: Свързване на градове с минимална цена

Условие: Дадени са n града и m възможни връзки (пътища) между тях. Всяка връзка (u, v, w) означава, че можем да свържем градове u и v с цена w . Намерете минималната цена за свързване на всички градове, или кажете, че е невъзможно.

Вход: n - брой градове, m - брой възможни връзки, списък от ребра (u_i, v_i, w_i) .

Изход: Минималната цена за свързване на всички градове, или -1 ако е невъзможно.

Идея: Това е директно приложение на MST. Градовете са върховете, а възможните пътища са ребрата. Графът е разреден (m може да е значително по-малко от n^2), затова Kruskal е подходящ. Ако след Kruskal имаме по-малко от $n - 1$ ребра в MST, значи графът не е свързан и е невъзможно.

Стъпки

1. Сортираме всички m ребра по тегло в нарастващ ред. Сложност: $O(m \cdot \log m)$.
2. Обхождаме ребрата в този ред. За всяко ребро (u, v, w) проверяваме чрез Union-Find дали u и v са в различни компоненти. Ако да — добавяме реброто към MST и обединяваме компонентите. Ако не — пропускаме го (би създавало цикъл). Сложност: $O(m)$.
3. Ако сме добавили точно $n - 1$ ребра, сумата от теглата им е отговорът. Ако след обхождането на всички ребра имаме по-малко от $n - 1$ — графът не е свързан и връщаме -1 .

Коректност

Kruskal разглежда ребрата в нарастващ ред по тегло и добавя само тези, които свързват два различни компонента. По Cut Property, всяко такова ребро принадлежи на MST. Ако не можем да съберем $n - 1$ ребра, значи графът не е свързан и покриващо дърво не съществува.

Времева сложност

$$\text{Time} \in O(m \cdot \log m) = O(m \cdot \log n)$$

Доминирана от сортирането. Забележка: $\log m = O(\log n)$, тъй като $m \leq n^2$.

Задача 3: Плаване на кораб между езера. Автор: Скелета

Условие: Имаме n езера свързани с m канала. Всеки канал i свързва две езера и има ширина w_i . Имаме кораб с ширина W , който започва в езеро s . Корабът може да мине през канал само ако ширината на канала е $\geq W$. Да се определи дали корабът може да стигне от езеро s до всички останали езера.

Вход: n - брой езера, m - брой канали, списък от канали (u_i, v_i, w_i) , начално езеро s , ширина на кораба W .

Изход: `true` ако корабът може да стигне от s до всички останали езера, `false` иначе.

Идея: Тук не се нуждаем от MST, а от **максимално покриващо дърво** (Maximum Spanning Tree). Защо? Искаме път от s до всеки друг връх, който максимизира **минималната ширина** по пътя (тъй като корабът е ограничен от най-тесния канал на пътя). Максималното покриващо дърво гарантира, че за всяка двойка върхове, пътят в дървото е **максимин пътят** - пътят, по който минималното тегло е максимално възможно.

Можем да използваме алгоритъма на Kruskal, но вместо да сортираме по нарастващо тегло, сортираме по **намаляващо тегло**. Така строим максимално покриващо дърво. След това проверяваме дали всички ребра в дървото имат ширина $\geq W$.

Алтернативно, по-просто решение: можем да филтрираме само каналите с ширина $\geq W$ и да проверим дали графът остава свързан (чрез BFS/DFS). Но решението с максимално покриващо дърво е по-общо и позволява да отговорим на въпроса за **всяка** ширина W .

Стъпки

1. Сортираме всички m канала по ширина в намаляващ ред. Сложност: $O(m \cdot \log m)$.
2. Прилагаме Kruskal, но в обратен ред — добавяме ребра от най-широкото към най-тесното, пропускайки тези, които създават цикъл (чрез Union-Find). Така строим максимално покриващо дърво. Сложност: $O(m)$.
3. Ако не успеем да съберем $n - 1$ ребра, графът не е свързан — корабът не може да стигне навсякъде, връщаме **false**.
4. Намираме минималната ширина сред ребрата на полученото дърво — това е максималната допустима ширина на кораб, който може да обходи всички езера. Ако W е по-малко или равно на тази минимална ширина, връщаме **true**, иначе **false**.

Коректност

Максималното покриващо дърво се строи аналогично на MST, но сортираме по намаляващо тегло. По Cut Property (приложена за максимум вместо минимум), на всяка стъпка избираме най-широкия канал, пресичащ среза — това гарантира, че пътят в дървото между всяка двойка върхове максимизира минималната ширина. Минималното ребро в дървото определя най-тесното “гърло” при пълно обхождане. Ако корабът е по-тесен от това гърло ($W \leq \text{minWidth}$), той може да стигне навсякъде.

Времева сложност

$$\text{Time} \in O(m \cdot \log m) = O(m \cdot \log n)$$

Доминирана от сортирането. Забележка: $\log m = O(\log n)$, тъй като $m \leq n^2$.